Class FileSystem

- java.lang.Object
- ◦ java.nio.file.FileSystem

- All Implemented Interfaces:
  Closeable, AutoCloseable

```
public abstract class FileSystem
extends Object
implements Closeable
```

Provides an interface to a file system and is the factory for objects to access files and other objects in the file system.

The default file system, obtained by invoking the `FileSystems.getDefault` method, provides access to the file system that is accessible to the Java virtual machine. The `FileSystems` class defines methods to create file systems that provide access to other types of (custom) file systems.

A file system is the factory for several types of objects:

- The `getPath` method converts a system dependent *path string*, returning a `Path` object that may be used to locate and access a file.
- The `getPathMatcher` method is used to create a `PathMatcher` that performs match operations on paths.
- The `getFileStores` method returns an iterator over the underlying `file-stores`.
- The `getUserPrincipalLookupService` method returns the `UserPrincipalLookupService` to lookup users or groups by name.
- The `newWatchService` method creates a `WatchService` that may be used to watch objects for changes and events.

File systems vary greatly. In some cases the file system is a single hierarchy of files with one top-level root directory. In other cases it may have several distinct file hierarchies, each with its own top-level root directory. The `getRootDirectories` method may be used to iterate over the root directories in the file system. A file system is typically composed of one or more underlying `file-stores` that provide the storage for the files. Theses file stores can also vary in the features they support, and the file attributes or *meta-data* that they associate with files.

A file system is open upon creation and can be closed by invoking its `close` method. Once closed, any further attempt to access objects in the file system cause `ClosedFileSystemException` to be thrown. File systems created by the default `provider` cannot be closed.

A `FileSystem` can provide read-only or read-write access to the file system. Whether or not a file system provides read-only access is established when the `FileSystem` is created and can be tested by invoking its `isReadOnly` method. Attempts to write to file stores by means of an object associated with a read-only file system throws `ReadOnlyFileSystemException`.

File systems are safe for use by multiple concurrent threads. The `close` method may be invoked at any time to close a file system but whether a file system is *asynchronously closeable* is provider specific and therefore unspecified. In other words, if a thread is accessing an object

in a file system, and another thread invokes the `close` method then it may require to block until the first operation is complete. Closing a file system causes all open channels, watch services, and other closeable objects associated with the file system to be closed.

Since:
1.7

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**FileSystem()**	Initializes a new instance of this class.

Method Summary

All Methods Instance Methods Abstract Methods

Modifier and Type	Method
abstract void	**close()**
abstract Iterable<FileStore>	**getFileStores()**
abstract Path	**getPath(String first, String... more)**
abstract PathMatcher	**getPathMatcher(String syntaxAndPattern)**
abstract Iterable<Path>	**getRootDirectories()**
abstract String	**getSeparator()**
abstract UserPrincipalLookupService	**getUserPrincipalLookupService()**
abstract boolean	**isOpen()**
abstract boolean	**isReadOnly()**
abstract WatchService	**newWatchService()**

abstract FileSystemProvider	**provider**()
abstract Set<String>	**supportedFileAttributeViews**()

- ### Methods inherited from class java.lang.Object

  clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

FileSystem

```
protected FileSystem()
```

Initializes a new instance of this class.

Method Detail

provider

```
public abstract FileSystemProvider provider()
```

Returns the provider that created this file system.
Returns:
The provider that created this file system.

close

```
public abstract void close()
                     throws IOException
```

Closes this file system.

After a file system is closed then all subsequent access to the file system, either by methods defined by this class or on objects associated with this file system, throw ClosedFileSystemException. If the file system is already closed then invoking this method has no effect.

Closing a file system will close all open channels, directory-streams, watch-service, and other closeable objects associated with this file system. The default file system cannot be closed.

Specified by:

close in interface Closeable

Specified by:

close in interface AutoCloseable

Throws:

IOException - If an I/O error occurs

UnsupportedOperationException - Thrown in the case of the default file system

■ isOpen

```
public abstract boolean isOpen()
```

Tells whether or not this file system is open.

File systems created by the default provider are always open.

Returns:

`true` if, and only if, this file system is open

■ isReadOnly

```
public abstract boolean isReadOnly()
```

Tells whether or not this file system allows only read-only access to its file stores.

Returns:

`true` if, and only if, this file system provides read-only access

■ getSeparator

```
public abstract String getSeparator()
```

Returns the name separator, represented as a string.

The name separator is used to separate names in a path string. An implementation may support multiple name separators in which case this method returns an implementation specific *default* name separator. This separator is used when creating path strings by invoking the `toString()` method.

In the case of the default provider, this method returns the same separator as `File.separator`.

Returns:

The name separator

■ getRootDirectories

```
public abstract Iterable<Path> getRootDirectories()
```

Returns an object to iterate over the paths of the root directories.

A file system provides access to a file store that may be composed of a number of distinct file hierarchies, each with its own top-level root directory. Unless denied by the security manager, each element in the returned iterator corresponds to the root directory of a distinct file hierarchy. The order of the elements is not defined. The file hierarchies may change during the lifetime of the Java virtual machine. For example, in some implementations, the insertion of removable media may result in the creation of a new file hierarchy with its own top-level directory.

When a security manager is installed, it is invoked to check access to the each root directory. If denied, the root directory is not returned by the iterator. In the case of the default provider, the `SecurityManager.checkRead(String)` method is invoked to check read access to each root directory. It is system dependent if the permission checks are done when the iterator is obtained or during iteration.

Returns:
An object to iterate over the root directories

- **getFileStores**

```
public abstract Iterable<FileStore> getFileStores()
```

Returns an object to iterate over the underlying file stores.

The elements of the returned iterator are the `FileStores` for this file system. The order of the elements is not defined and the file stores may change during the lifetime of the Java virtual machine. When an I/O error occurs, perhaps because a file store is not accessible, then it is not returned by the iterator.

In the case of the default provider, and a security manager is installed, the security manager is invoked to check `RuntimePermission("getFileStoreAttributes")`. If denied, then no file stores are returned by the iterator. In addition, the security manager's `SecurityManager.checkRead(String)` method is invoked to check read access to the file store's *top-most* directory. If denied, the file store is not returned by the iterator. It is system dependent if the permission checks are done when the iterator is obtained or during iteration.

Usage Example: Suppose we want to print the space usage for all file stores:

```
for (FileStore store: FileSystems.getDefault().getFileSto
    long total = store.getTotalSpace() / 1024;
    long used = (store.getTotalSpace() - store.getUnallo
    long avail = store.getUsableSpace() / 1024;
    System.out.format("%-20s %12d %12d %12d%n", store, to
}
```

Returns:
An object to iterate over the backing file stores

■ supportedFileAttributeViews

```
public abstract Set<String> supportedFileAttributeViews()
```

Returns the set of the `names` of the file attribute views supported by this `FileSystem`.

The `BasicFileAttributeView` is required to be supported and therefore the set contains at least one element, "basic".

The `supportsFileAttributeView(String)` method may be used to test if an underlying `FileStore` supports the file attributes identified by a file attribute view.

Returns:
An unmodifiable set of the names of the supported file attribute views

■ getPath

```
public abstract Path getPath(String first,
                             String... more)
```

Converts a path string, or a sequence of strings that when joined form a path string, to a `Path`. If `more` does not specify any elements then the value of the `first` parameter is the path string to convert. If `more` specifies one or more elements then each non-empty string, including `first`, is considered to be a sequence of name elements (see `Path`) and is joined to form a path string. The details as to how the Strings are joined is provider specific but typically they will be joined using the `name-separator` as the separator. For example, if the name separator is `"/"` and `getPath("/foo","bar","gus")` is invoked, then the path string `"/foo/bar/gus"` is converted to a `Path`. A `Path` representing an empty path is returned if `first` is the empty string and `more` does not contain any non-empty strings.

The parsing and conversion to a path object is inherently implementation dependent. In the simplest case, the path string is rejected, and `InvalidPathException` thrown, if the

path string contains characters that cannot be converted to characters that are *legal* to the file store. For example, on UNIX systems, the NUL (\u0000) character is not allowed to be present in a path. An implementation may choose to reject path strings that contain names that are longer than those allowed by any file store, and where an implementation supports a complex path syntax, it may choose to reject path strings that are *badly formed*.

In the case of the default provider, path strings are parsed based on the definition of paths at the platform or virtual file system level. For example, an operating system may not allow specific characters to be present in a file name, but a specific underlying file store may impose different or additional restrictions on the set of legal characters.

This method throws `InvalidPathException` when the path string cannot be converted to a path. Where possible, and where applicable, the exception is created with an `index` value indicating the first position in the `path` parameter that caused the path string to be rejected.

Parameters:
`first` - the path string or initial part of the path string
`more` - additional strings to be joined to form the path string
Returns:
the resulting `Path`
Throws:
`InvalidPathException` - If the path string cannot be converted

- **getPathMatcher**

```
public abstract PathMatcher getPathMatcher(String syntaxAndPa
```

Returns a `PathMatcher` that performs match operations on the `String` representation of `Path` objects by interpreting a given pattern. The `syntaxAndPattern` parameter identifies the syntax and the pattern and takes the form:

```
syntax:pattern
```

where `':'` stands for itself.

A `FileSystem` implementation supports the "`glob`" and "`regex`" syntaxes, and may support others. The value of the syntax component is compared without regard to case.

When the syntax is "`glob`" then the `String` representation of the path is matched using a limited pattern language that

resembles regular expressions but with a simpler syntax. For example:

`*.java`	Matches a path that represents a file name ending in `.java`
`*.*`	Matches file names containing a dot
`*.{java,class}`	Matches file names ending with `.java` or `.class`
`foo.?`	Matches file names starting with `foo.` and a single character extension
`/home/*/*`	Matches `/home/gus/data` on UNIX platforms
`/home/**`	Matches `/home/gus` and `/home/gus/data` on UNIX platforms
`C:\\*`	Matches `C:\foo` and `C:\bar` on the Windows platform (note that the backslash is escaped; as a string literal in the Java Language the pattern would be `"C:\\\\*"`)

The following rules are used to interpret glob patterns:

- The * character matches zero or more `characters` of a `name` component without crossing directory boundaries.
- The ** characters matches zero or more `characters` crossing directory boundaries.
- The ? character matches exactly one character of a name component.
- The backslash character (\) is used to escape characters that would otherwise be interpreted as special characters. The expression \\ matches a single backslash and "\{" matches a left brace for example.
- The `[ ]` characters are a *bracket expression* that match a single character of a name component out of a set of characters. For example, `[abc]` matches `"a"`, `"b"`, or `"c"`. The hyphen (`-`) may be used to specify a range so `[a-z]` specifies a range that matches from `"a"` to `"z"` (inclusive). These forms can be mixed so [abce-g] matches `"a"`, `"b"`, `"c"`, `"e"`, `"f"` or `"g"`. If the character after the `[` is a `!` then it is used for negation so `[!a-c]` matches any character except `"a"`, `"b"`, or `"c"`.

  Within a bracket expression the *, ? and \ characters match themselves. The (`-`) character matches itself if it is

the first character within the brackets, or the first character after the `!` if negating.

- The `{ }` characters are a group of subpatterns, where the group matches if any subpattern in the group matches. The `","` character is used to separate the subpatterns. Groups cannot be nested.
- Leading period/dot characters in file name are treated as regular characters in match operations. For example, the `"*"` glob pattern matches file name `".login"`. The `Files.isHidden(java.nio.file.Path)` method may be used to test whether a file is considered hidden.
- All other characters match themselves in an implementation dependent manner. This includes characters representing any `name-separators`.
- The matching of `root` components is highly implementation-dependent and is not specified.

When the syntax is `"regex"` then the pattern component is a regular expression as defined by the `Pattern` class.

For both the glob and regex syntaxes, the matching details, such as whether the matching is case sensitive, are implementation-dependent and therefore not specified.

Parameters:
`syntaxAndPattern` - The syntax and pattern
Returns:
A path matcher that may be used to match paths against the pattern
Throws:
`IllegalArgumentException` - If the parameter does not take the form: `syntax:pattern`
`PatternSyntaxException` - If the pattern is invalid
`UnsupportedOperationException` - If the pattern syntax is not known to the implementation
See Also:
`Files.newDirectoryStream(Path,String)`

- **getUserPrincipalLookupService**

```
public abstract UserPrincipalLookupService getUserPrincipalLo
```

Returns the `UserPrincipalLookupService` for this file system *(optional operation)*. The resulting lookup service may be used to lookup user or group names.

Usage Example: Suppose we want to make "joe" the owner of a file:

```
UserPrincipalLookupService lookupService = FileSystems.ge
Files.setOwner(path, lookupService.lookupPrincipalByName
```

Returns:
The `UserPrincipalLookupService` for this file system
Throws:
`UnsupportedOperationException` - If this `FileSystem` does not does have a lookup service

■ newWatchService

```
public abstract WatchService newWatchService()
                                          throws IOException
```

Constructs a new `WatchService` *(optional operation)*.

This method constructs a new watch service that may be used to watch registered objects for changes and events.

Returns:
a new watch service
Throws:
`UnsupportedOperationException` - If this `FileSystem` does not support watching file system objects for changes and events. This exception is not thrown by `FileSystems` created by the default provider.
`IOException` - If an I/O error occurs

- Field |
- Constr |
- Method

- Detail:
- Field |
- Constr |
- Method

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.